

AI Assisted Coding Lab ASS-17.1

G. SANJAY

2303A510B9

B-13

Task 1 – Customer Data Cleaning

Task:

Use AI to generate a Python script for cleaning a customer dataset.

PROMPT: Generate a Python script using Pandas to clean a customer dataset loaded from a CSV file named "shopping_trends.csv". The script should handle missing values in columns such as age, income, and city by filling or removing them appropriately. It should remove duplicate records from the dataset. Convert all text values in the city column to lowercase to ensure consistency. Encode categorical variables like gender and city into numerical format using suitable encoding techniques. Finally, display the cleaned dataset and ensure it is ready for further data analysis.

```
[1] import pandas as pd
    from sklearn.preprocessing import LabelEncoder

    # Load the dataset
    try:
        df = pd.read_csv('shopping_trends.csv')
        print("Dataset loaded successfully.")
    except FileNotFoundError:
        print("Error: 'shopping_trends.csv' not found. Please make sure the file is in the correct directory.")
        df = pd.DataFrame() # Create an empty DataFrame to avoid errors in subsequent steps

    # Display initial info and head to understand the data
    if not df.empty:
        print("\nInitial dataset info:")
        df.info()
        print("\nFirst 5 rows of the original dataset:")
        display(df.head())

--- Error: 'shopping_trends.csv' not found. Please make sure the file is in the correct directory.

[2] if not df.empty:
    print("\nMissing values before handling:")
    print(df[['Age', 'Income', 'City']].isnull().sum())

    # Remove rows with missing values in 'Age', 'Income', or 'City'
    initial_rows = df.shape[0]
    df.dropna(subset=['Age', 'Income', 'City'], inplace=True)
    rows_after_na = df.shape[0]
    print(f"\nRemoved {initial_rows - rows_after_na} rows with missing values in Age, Income, or City.")

    print("\nMissing values after handling:")
    print(df[['Age', 'Income', 'City']].isnull().sum())

[3] if not df.empty:
    initial_rows = df.shape[0]
    df.drop_duplicates(inplace=True)
    rows_after_duplicates = df.shape[0]
    print(f"\nRemoved {initial_rows - rows_after_duplicates} duplicate rows.")
    print(f"Dataset now has {df.shape[0]} rows.")

[4] if not df.empty and 'City' in df.columns:
    df['City'] = df['City'].str.lower()
    print("\n'n'City' column converted to lowercase.")
    display(df[['City']].head())
```

```
[3] ✓ Os
if not df.empty:
    initial_rows = df.shape[0]
    df.drop_duplicates(inplace=True)
    rows_after_duplicates = df.shape[0]
    print(f"\nRemoved {initial_rows - rows_after_duplicates} duplicate rows.")
    print(f"Dataset now has {df.shape[0]} rows.")

[4] ✓ Os
if not df.empty and 'City' in df.columns:
    df['City'] = df['City'].str.lower()
    print("\n'City' column converted to lowercase.")
    display(df[['City']].head())

[6] ✓ Os
if not df.empty:
    # Encode 'Gender'
    if 'Gender' in df.columns:
        gender_mapping = {'Male': 0, 'Female': 1}
        df['Gender_Encoded'] = df['Gender'].map(gender_mapping)
        print("\n'Gender' column encoded.")
        print("Original Gender unique values:", df['Gender'].unique())
        print("Encoded Gender unique values:", df['Gender_Encoded'].unique())

    # Encode 'City'
    if 'City' in df.columns:
        le = LabelEncoder()
        df['City_Encoded'] = le.fit_transform(df['City'])
        print("\n'City' column encoded.")
        print("Original City unique values (first 5):", df['City'].unique()[:5])
        print("Encoded City unique values (first 5):", df['City_Encoded'].unique()[:5])

    print("\nCleaned dataset info:")
    df.info()
    print("\nFirst 5 rows of the cleaned dataset:")
    display(df.head())

The dataset has been cleaned and is now ready for further analysis!

[9] ✓ Os
!unzip -o '/archive (8).zip' -d '.'

Archive: /archive (8).zip
  inflating: ./shopping_trends.csv
  inflating: ./shopping_trends_updated.csv
```

Task 2 – Sales Data Preprocessing

Task:

Preprocess sales transaction data using AI assistance.

PROMPT: Generate a Python script using Pandas and NumPy to preprocess a sales dataset loaded from a CSV file named "sales_data.csv". Convert the date column into datetime format and extract new features such as month, year, and day of the week. Normalize the sales amount column using an appropriate scaling method like Min-Max normalization. Detect outliers in the dataset using techniques such as the IQR method or Z-score and handle them by removing or capping extreme values. Finally, display the transformed dataset with the new features, ensuring it is ready for further analysis

```
import pandas as pd

# Assuming the file is uploaded to the Cloud environment
# If you placed it in /tmp/, change the path accordingly
df = pd.read_csv('sales_data.csv')

display(df.head())

# Import numpy as np
from sklearn.preprocessing import MinMaxScaler

# Convert 'date' column to datetime format
df['date'] = pd.to_datetime(df['date'])
# Convert 'sale_date' column to datetime format
df['sale_date'] = pd.to_datetime(df['sale_date'])

# Extract new features: month, year, day of the week
df['month'] = df['date'].dt.month
df['year'] = df['date'].dt.year
df['day_of_week'] = df['date'].dt.dayofweek + 1 # Monday=0, Sunday=6
df['month'] = df['sale_date'].dt.month
df['year'] = df['sale_date'].dt.year
df['day_of_week'] = df['sale_date'].dt.dayofweek + 1 # Monday=0, Sunday=6

print('New features extracted:')
display(df.head())

# Data features extracted:
Product_ID Sale_Date Sales_Rep Region Sales_Amount Quantity_Sold Product_Category Unit_Cost Unit_Price Customer_Type Discount Payment_Method Sales_Channel Region_and_Sales_Rep month year day_of_week
0 1002 2023-03-03 Bob North 5053.97 18 Furniture 152.75 267.22 Returning 0.09 Cash Online North-Bob 2 2023 4
1 1003 2023-04-21 Bob West 4384.02 17 Furniture 3816.36 4209.44 Returning 0.11 Cash Retail West-Bob 4 2023 4
2 1015 2023-09-21 David South 4031.23 30 Food 261.56 371.40 Returning 0.20 Bank Transfer Retail South-David 9 2023 3
3 1072 2023-06-24 Bob South 2167.94 39 Clothing 4330.03 4467.75 New 0.02 Credit Card Retail South-Bob 6 2023 3
4 1081 2023-03-24 Charlie East 3750.20 13 Electronics 637.37 692.71 New 0.06 Credit Card Online East-Charlie 3 2023 4

# scaler = MinMaxScaler()
# scaler.fit(df[['Sales_Amount', 'Unit_Cost', 'Unit_Price', 'Discount', 'Quantity_Sold']])
# df[['Sales_Amount', 'Unit_Cost', 'Unit_Price', 'Discount', 'Quantity_Sold']] = scaler.transform(df[['Sales_Amount', 'Unit_Cost', 'Unit_Price', 'Discount', 'Quantity_Sold']])
```

```
scaler = MinMaxScaler()
df[['sale_amount_normalized']] = scaler.fit_transform(df[['sale_amount']])
df[['unit_cost_normalized']] = scaler.fit_transform(df[['unit_cost']])
df[['unit_price_normalized']] = scaler.fit_transform(df[['unit_price']])
df[['discount_normalized']] = scaler.fit_transform(df[['discount']])
df[['quantity_sold_normalized']] = scaler.fit_transform(df[['quantity_sold']])

print('Sales amount normalized:')
display(df.head())

# Sales amount normalized:
Product_ID Sale_Date Sales_Rep Region Sales_Amount Quantity_Sold Product_Category Unit_Cost Unit_Price Customer_Type Discount Payment_Method Sales_Channel Region_and_Sales_Rep month year day_of_week sale_amount_normalized
0 1002 2023-03-03 Bob North 5053.97 18 Furniture 152.75 267.22 Returning 0.09 Cash Online North-Bob 2 2023 4 0.000960
1 1003 2023-04-21 Bob West 4384.02 17 Furniture 3816.36 4209.44 Returning 0.11 Cash Retail West-Bob 4 2023 4 0.433002
2 1015 2023-09-21 David South 4031.23 30 Food 261.56 371.40 Returning 0.20 Bank Transfer Retail South-David 9 2023 3 0.409021
3 1072 2023-06-24 Bob South 2167.94 39 Clothing 4330.03 4467.75 New 0.02 Credit Card Retail South-Bob 6 2023 3 0.209195
4 1081 2023-03-24 Charlie East 3750.20 13 Electronics 637.37 692.71 New 0.06 Credit Card Online East-Charlie 3 2023 4 0.369108

# Calculate Q1 (25th percentile) and Q3 (75th percentile)
Q1 = df['sale_amount'].quantile(0.25)
Q3 = df['sale_amount'].quantile(0.75)
IQR = Q3 - Q1

# Define outlier bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Identify outliers
outliers = df[(df['sale_amount'] < lower_bound) | (df['sale_amount'] > upper_bound)]
print('Number of outliers detected: {}'.format(len(outliers)))

# Remove outliers
df = df[outliers == False]
print('Dataset size after removing outliers: {}'.format(df.shape[0]))

# Option 2: Cap extreme values (replace outliers with bounds)
df['sale_amount_capped'] = np.where(
    df['sale_amount'] < lower_bound, lower_bound,
    np.where(df['sale_amount'] > upper_bound, upper_bound, df['sale_amount'])
)
df['sale_amount_capped'] = df['sale_amount'].where(
    df['sale_amount'] > lower_bound & df['sale_amount'] < upper_bound,
    lower_bound, upper_bound, df['sale_amount']
)

print('Original sales_amount vs capped sales_amount for some rows (including potential outliers):')
display(df[['sale_amount', 'sale_amount_capped']].sample(10))

# Number of outliers detected & Original sales_amount vs capped sales_amount for some rows (including potential outliers):
sale_amount sale_amount_capped
0 5053.97 5053.97
217 4422.46 4422.46
327 1726.16 1726.16
3 4031.23 4031.23
382 1942.75 1942.75

# Print final transformed dataset (first 5 rows):
display(df.head())

# Final transformed dataset (first 5 rows):
Product_ID Sale_Date Sales_Rep Region Sales_Amount Quantity_Sold Product_Category Unit_Cost Unit_Price Customer_Type Discount Payment_Method Sales_Channel Region_and_Sales_Rep month year day_of_week sale_amount_normalized sale_amount_capped
0 1002 2023-03-03 Bob North 5053.97 18 Furniture 152.75 267.22 Returning 0.09 Cash Online North-Bob 2 2023 4 0.000960 5053.97
1 1003 2023-04-21 Bob West 4384.02 17 Furniture 3816.36 4209.44 Returning 0.11 Cash Retail West-Bob 4 2023 4 0.433002 4384.02
2 1015 2023-09-21 David South 4031.23 30 Food 261.56 371.40 Returning 0.20 Bank Transfer Retail South-David 9 2023 3 0.409021 4031.23
3 1072 2023-06-24 Bob South 2167.94 39 Clothing 4330.03 4467.75 New 0.02 Credit Card Retail South-Bob 6 2023 3 0.209195 2167.94
4 1081 2023-03-24 Charlie East 3750.20 13 Electronics 637.37 692.71 New 0.06 Credit Card Online East-Charlie 3 2023 4 0.369108 3750.20
```

Task 3 – Healthcare Data Preparation Task:

Clean and preprocess a healthcare dataset using AI scripts.

PROMPT: Generate a Python script using Pandas and Scikit-learn to clean and preprocess a healthcare dataset loaded from a CSV file. The script should handle missing values in columns such as blood_pressure and cholesterol by filling or removing them appropriately. Encode categorical features using techniques like label encoding or one-hot encoding. Scale numerical features using methods such as Min-Max scaling or Standard scaling. Finally, split the dataset into training and testing sets using an appropriate ratio (e.g., 80:20) and display the processed data, ensuring it is ready for machine learning models.

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# Load the dataset (replace 'healthcare_data.csv' with your actual file name)
try:
    df = pd.read_csv('healthcare_data.csv')
    print("Dataset loaded successfully. First 5 rows:")
    display(df.head())
except FileNotFoundError:
    print("Error: 'healthcare_data.csv' not found. Please upload the file to your Colab environment or specify the correct path.")
    # Create a dummy DataFrame for demonstration if file not found
    data = {
        'Patient_ID': range(1, 11),
        'Age': [30, 45, 60, 25, 50, 35, 70, 40, 55, 65],
        'Gender': ['Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female'],
        'Blood_Pressure': [120, 140, None, 110, 130, 125, 150, 118, None, 135],
        'Cholesterol': [180, 220, 200, 170, None, 190, 240, 160, 210, None],
        'Smoker': ['Yes', 'No', 'No', 'Yes', 'No', 'No', 'Yes', 'No', 'Yes', 'No'],
        'Diagnosis': ['Healthy', 'High BP', 'High Chol', 'Healthy', 'High BP', 'Healthy', 'High BP', 'Healthy', 'High BP', 'Healthy']
    }
    df = pd.DataFrame(data)
    print("Using dummy data for demonstration.")
    display(df.head())
```

Error: 'healthcare_data.csv' not found. Please upload the file to your Colab environment or specify the correct path.
Using dummy data for demonstration.

Patient_ID	Age	Gender	Blood_Pressure	Cholesterol	Smoker	Diagnosis	
0	1	30	Male	120.0	180.0	Yes	Healthy
1	2	45	Female	140.0	220.0	No	High BP
2	3	60	Male	NaN	200.0	No	High Chol
3	4	25	Female	110.0	170.0	Yes	Healthy
4	5	50	Male	130.0	NaN	No	High BP

```
# Assuming 'Diagnosis' is the target variable
target_column = 'Diagnosis'

# Separate features (X) and target (y)
X = df.drop(columns=[target_column])
y = df[target_column]

# Identify numerical and categorical columns for preprocessing
numerical_cols = X.select_dtypes(include=np.number).columns.tolist()
categorical_cols = X.select_dtypes(include='object').columns.tolist()

print(f"Numerical columns: {numerical_cols}")
print(f"Categorical columns: {categorical_cols}")

Numerical columns: ['Patient_ID', 'Age', 'Blood_Pressure', 'Cholesterol']
Categorical columns: ['Gender', 'Smoker']
```

```

23) # Create preprocessing pipelines for numerical and categorical features
numerical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')), # Fill missing numerical values with the mean
    ('scaler', StandardScaler()) # Scale numerical features
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')), # Fill missing categorical values with the most frequent
    ('onehot', OneHotEncoder(handle_unknown='ignore')) # One-hot encode categorical features
])

# Combine preprocessing steps using ColumnTransformer
# This applies different transformers to different columns
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_cols),
        ('cat', categorical_transformer, categorical_cols)
    ]
)

# Create a full preprocessing pipeline
full_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor)
])

# Apply the preprocessing to the features
X_processed = full_pipeline.fit_transform(X)

# Convert processed features back to DataFrame (optional, but good for inspection)
# Get feature names after one-hot encoding
# Need to access the 'onehot' step within the 'cat' transformer of the preprocessor
encoded_feature_names = full_pipeline.named_steps['preprocessor'].named_transformers_['cat'].named_steps['onehot'].get_feature_names_out(categorical_cols)
new_feature_names = numerical_cols + encoded_feature_names.tolist()

X_processed_df = pd.DataFrame(X_processed, columns=new_feature_names)

print("Processed features (first 5 rows):")
display(X_processed_df.head())

```

Processed features (first 5 rows):

	Patient_ID	Age	Blood_Pressure	Cholesterol	Gender_Female	Gender_Male	Smoker_No	Smoker_Yes
0	-1.566699	-1.218544	-0.783618	-0.727632	0.0	1.0	0.0	1.0
1	-1.218544	-0.174078	1.060460	1.063462	1.0	0.0	1.0	0.0
2	-0.870388	0.870388	0.000000	0.167915	0.0	1.0	1.0	0.0
3	-0.522233	-1.566699	-1.705957	-1.175406	1.0	0.0	0.0	1.0
4	-0.174078	0.174078	0.138321	0.000000	0.0	1.0	1.0	0.0

```

# Split the processed data into training and testing sets (80:20 ratio)
X_train, X_test, y_train, y_test = train_test_split(
    X_processed_df, y, test_size=0.3, random_state=42, stratify=y # stratify helps maintain class distribution
)

print(f"Shape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")

print("\nFirst 5 rows of X_train:")
display(X_train.head())

print("\nFirst 5 rows of y_train:")
display(y_train.head())

```

Shape of X_train: (7, 8)
Shape of X_test: (3, 8)
Shape of y_train: (7,)
Shape of y_test: (3,)

First 5 rows of X_train:

	Patient_ID	Age	Blood_Pressure	Cholesterol	Gender_Female	Gender_Male	Smoker_No	Smoker_Yes
4	-0.174078	0.174078	0.138321	0.000000	0.0	1.0	1.0	0.0
8	1.218544	0.522233	0.000000	0.615689	0.0	1.0	0.0	1.0
3	-0.522233	-1.566699	-1.705957	-1.175406	1.0	0.0	0.0	1.0
9	1.566699	1.218544	0.599390	0.000000	1.0	0.0	1.0	0.0
1	-1.218544	-0.174078	1.060460	1.063462	1.0	0.0	1.0	0.0

First 5 rows of y_train:

	Diagnosis
4	High BP
8	High Chol
3	Healthy
9	High BP
1	High BP

dtype: object

Task 4 – Employee Salary Data Preprocessing

Task:

Clean and preprocess an employee salary dataset using AI scripts.

Instructions

PROMPT: Generate a Python script using Pandas and Scikit-learn to clean and preprocess an employee salary dataset loaded from a CSV file. The script should handle missing values by filling or removing them appropriately. Detect and remove salary outliers using techniques such as the IQR method or Z-score. Standardize text values in the department column by converting them to lowercase for consistency. Apply label encoding to categorical columns such as job location and department. Finally, display the cleaned and structured dataset, ensuring it is ready for HR analytics.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder

# Load the dataset (replace 'Employee_Salary_Dataset.csv' with your actual file name)
try:
    df = pd.read_csv('Employee_Salary_Dataset.csv')
    print('Dataset loaded successfully. First 5 rows:')
    display(df.head())
except FileNotFoundError:
    print('Error: Employee_Salary_Dataset.csv not found. Please upload the file or specify the correct path.')
    # Create a dummy dataframe for demonstration if file not found
    data = {
        'Employee_ID': range(1, 11),
        'Age': [25, 30, 35, 40, 28, 45, 50, 32, 38, 20],
        'Gender': ['Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female'],
        'Department': ['Sales', 'Marketing', 'IT', 'HR', 'Sales', 'Finance', 'IT', 'Marketing', 'HR', 'Sales'],
        'Job_Title': ['Sales Rep', 'Marketing Analyst', 'Software Engineer', 'HR Manager', 'Sales Manager', 'Financial Analyst', 'Data Scientist', 'Marketing Specialist', 'HR Coordinator', 'Sales Associate'],
        'Job_Location': ['New York', 'London', 'Remote', 'New York', 'London', 'Remote', 'New York', 'London', 'Remote', 'New York'],
        'Years_of_Experience': [2, 5, 8, 12, 4, 15, 10, 6, 9, 1],
        'Salary': [50000, 60000, 80000, 90000, 70000, 100000, 95000, 65000, 75000, 55000] # Added a potential outlier at 150k and a NaN
    }
    df = pd.DataFrame(data)
    print('Using dummy data for demonstration.')
    display(df.head())

# Display initial info
print('Initial DataFrame Info:')
df.info()

-- Dataset loaded successfully. First 5 rows:
   ID  Experience_Years  Age  Gender  Salary
0  1                5   28  Female  250000
1  2                1   21   Male   50000
2  3                3   23  Female  170000
3  4                2   22   Male   25000
4  5                1   17   Male  120000

Initial DataFrame Info:
Out[1]: pandas.core.frame.DataFrame
RangeIndex: 35 entries, 0 to 34
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  --
 0   ID            35 non-null      int64
 1   Experience_Years  35 non-null      int64
 2   Age           35 non-null      int64
 3   Gender        35 non-null      object
 4   Salary        35 non-null      int64
dtypes: int64(4), object(1)
memory usage: 1.5+ KB
```



```
# Fill missing 'Salary' values with the median
if 'Salary' in df.columns:
    median_salary = df['Salary'].median()
    df['Salary'].fillna(median_salary, inplace=True)
    print(f"Filled missing 'Salary' values with median: {median_salary}")

# For simplicity, if any other crucial columns have NaNs, we'll drop those rows.
# This might need to be adjusted based on the specific dataset and domain knowledge.
df.dropna(inplace=True)

print("DataFrame after handling missing values:")
df.info()
```

— Filled missing 'Salary' values with median: 250000.0

DataFrame after handling missing values:

```
class 'pandas.core.frame.DataFrame'
RangeIndex: 35 entries, 0 to 34
Data columns (total 5 columns):
 #   Column          Non-Null Count  Dtype
---  --
 0   ID              35 non-null      int64
 1   Experience_Years 35 non-null      int64
 2   Age             35 non-null      int64
 3   Gender          35 non-null      object
 4   Salary          35 non-null      int64
dtypes: int64(4), object(1)
memory usage: 1.5+ KB
```

/tmp/ipykernel_355/3865862158.py:4: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy. For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['Salary'].fillna(median_salary, inplace=True)
```

2. Detect and Remove Salary Outliers (IQR Method)

We'll use the Interquartile Range (IQR) method to identify and remove outliers in the 'Salary' column.

```
if 'Salary' in df.columns:
    Q1 = df['Salary'].quantile(0.25)
    Q3 = df['Salary'].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Identify outliers
    outliers = df[(df['Salary'] < lower_bound) | (df['Salary'] > upper_bound)]
    print(f"Number of salary outliers detected: {len(outliers)}")
    if not outliers.empty:
        print("Salary Outliers:")
        display(outliers)

    # Remove outliers
    df = df[~(df['Salary'] < lower_bound) | (df['Salary'] > upper_bound)]
    print("DataFrame after removing salary outliers:")
    display(df.head())
else:
    print("No 'Salary' column found for outlier detection.")
```

Number of salary outliers detected: 2
Salary Outliers:

ID	Experience_Years	Age	Gender	Salary
27	28	27	Female	10000000
22	34	19	Female	9300000

DataFrame after removing salary outliers:

ID	Experience_Years	Age	Gender	Salary	
0	1	5	28	Female	250000
1	2	1	21	Male	50000
2	3	3	23	Female	170000
3	4	2	22	Male	25000
4	5	1	17	Male	10000

3. Standardize Text Values in 'Department' Column

Convert all department names to lowercase for consistency.

```
if 'Department' in df.columns:
    df['Department'] = df['Department'].str.lower()
    print("\n'Department' column after standardization (lowercase):")
    print(df['Department'].value_counts())
else:
    print("No 'Department' column found for standardization.")
```

No 'Department' column found for standardization.

4. Apply Label Encoding to Categorical Columns

We'll encode 'Gender', 'Department', and 'Job_Location' using `LabelEncoder`.

```
categorical_cols_to_encode = ['Gender', 'Department', 'Job_Location']

for col in categorical_cols_to_encode:
    if col in df.columns:
        le = LabelEncoder()
        df[col + '_Encoded'] = le.fit_transform(df[col])
        print(f'\nLabel encoded '{col}':')
        display(df[[col, col + '_Encoded']].drop_duplicates().sort_values(by=col + '_Encoded'))
    else:
        print(f'Column '{col}' not found for encoding:')

print("\nFinal Cleaned and Preprocessed Dataset (first 5 rows):")
display(df.head())

print("\nFinal DataFrame Info:")
df.info()
```

```
***
Label encoded 'Gender':
   Gender  Gender_Encoded
0  Female             0
1   Male             1
Column 'Department' not found for encoding.
Column 'Job_Location' not found for encoding.

Final Cleaned and Preprocessed Dataset (first 5 rows):
   ID  Experience_Years  Age  Gender  Salary  Gender_Encoded
0   1                 5   28  Female  250000             0
1   2                 1   21   Male    50000             1
2   3                 3   23  Female  170000             0
3   4                 2   22   Male   25000             1
4   5                 1   17   Male   10000             1

Final DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
Index: 33 entries, 0 to 34
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   ID              33 non-null    int64
1   Experience_Years 33 non-null    int64
2   Age             33 non-null    int64
3   Gender          33 non-null    object
4   Salary          33 non-null    int64
5   Gender_Encoded  33 non-null    int64
dtypes: int64(5), object(1)
memory usage: 1.8+ KB
```